GenAI - Code is on the last page in a table
ChatGPT - Free version, no think mode
Gemini - Free version using the 3.6 Flash version that was defaulted
It is important to consider that turning on modes for each GenAI may have resulted in different results as it makes the AI think longer thus coming up with different solutions,
Both were accessed on their respective website i.e https://chatgpt.com/ and https://gemini.google.com/app

Prompt:

Write a complete C++ program in a single file named Assignment4.cpp that solves 9x9 Sudoku puzzles using an object-oriented approach with classes and objects, rather than standalone functions. The program must use a depth-first search algorithm with backtracking. It should read Sudoku puzzle files where blanks are represented by underscores (_), solve them, and print out the puzzle file name, the initial puzzle, and all found solutions (or "No solution found" if none exist). Include clean, structured object-oriented design and detailed comments.

Correctness

- Both correctly implement DFS backtracking and object encapsulation, but ChatGPT's file-parsing logic handles arbitrary whitespace and newlines more robustly via file.get(ch) combined with a vector filter, whereas Gemini's stream extraction assumes a standard grid formatting. ChatGPT also includes a validation check for initial clues (hasValidInitialClues).
- Gemini uses two separate classes (SudokuBoard and SudokuSolver), which adheres strictly to object-oriented principles by decoupling board state/rules from the solving algorithm.
- ChatGPT encapsulates everything into a single Sudoku class. While compact, Gemini's separation is cleaner from a software engineering design perspective.
- Gemini's main function currently only grabs argv[1], meaning it processes a single file per execution unless modified.
- ChatGPT's main function loops through argc so it can handle multiple files passed via command line (e.g., Assignment4.exe puzzle1.txt puzzle2.txt puzzle3.txt).
-

execution time

- Both feature O(9^n) worst-case time complexity for DFS, executing well within the required fraction of a second on standard hardware and Cycle servers.

Space Complexity

- They both have the same

Maintainability

- Gemini collects all valid solutions into a std::vector<SudokuBoard> and prints them out at the end. This is great if you want to inspect or manipulate solutions post-search.
- ChatGPT prints solutions immediately as they are discovered during the recursive traversal.
-

My Fixes
Decision: I will decide to use Gemini for the source code.
Correctness

- Modify the Gemini code so that instead of only taking one file, assisted by gemini I should be able to make a loop to be able to add all of the files.

execution time

- I can't come up with a solution to the execution time, however by making a loop to get all the required files, It should be faster than getting them one by one.

Space Complexity

- The same space complexity as before the change, but adding a new SudokuPuzzle inside each loop makes sure that each one is separate and doesn't leak to anything outside of its own loop.

Maintainability

- Just adding overall more comments including documentation and adding some helper methods like solve or printboard.

ChatGPT	Gemini

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

/*
 * Assignment4.cpp
 *
 * Description:
 * Solves 9x9 Sudoku puzzles using depth-first search (DFS)
 * and backtracking.
 *
 * Input format:
 * - Each puzzle contains exactly 81 cells.
 * - Digits 1-9 represent filled cells.
 * - Underscores (_) represent blank cells.
 * - Whitespace and line breaks are ignored.
 *
 * Usage:
 *   Assignment4.exe puzzle1.txt puzzle2.txt
 *
 * If no filenames are provided as command-line arguments,
 * the program prompts the user to enter filenames.
 */

// Represents and solves a single Sudoku puzzle.
class Sudoku {
private:
    static const int SIZE = 9;

    // Stores the original puzzle and the working grid.
    int original[SIZE][SIZE];
    int grid[SIZE][SIZE];

    // Counts the number of solutions discovered.
    unsigned long long solutionCount;

    // Reads the puzzle from a file.
    bool loadFromFile(const string& filename) {
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            cerr << "Error: Could not open file: "
                 << filename << endl;
            return false;
```

```
/**
 * @file Assignment4.cpp
 * @brief Object-oriented 9x9 Sudoku Solver using Depth-First Search (DFS) with Backtracking.
 *
 * This program reads a 9x9 Sudoku puzzle from a text file where empty cells are
 * represented by underscores ('_'), solves the puzzle finding all possible solutions,
 * and prints out the file name, initial puzzle, and all valid solutions.
 */

#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <iomanip>

/**
 * @class SudokuBoard
 * @brief Represents a 9x9 Sudoku grid and handles board operations and rule validation.
 */
class SudokuBoard {
private:
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Board object initialized with empty cells ('_').
     */
    SudokuBoard() : grid(9, std::vector<char>(9, '_')) {}

    /**
     * @brief Loads a 9x9 Sudoku puzzle from a file.
     * @param filename The path to the puzzle text file.
     * @return true if successfully loaded exactly 81 valid characters, false otherwise.
     */
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        char ch;
        int count = 0;
        // Read characters sequentially, ignoring whitespace
```

```
    }

    vector<char> cells;
    char ch;

    // Extract digits and underscores while ignoring
    // whitespace and other formatting characters.
    while (inputFile.get(ch)) {
        if ((ch >= '1' && ch <= '9') || ch == '_') {
            cells.push_back(ch);
        }
    }

    inputFile.close();

    // A standard Sudoku puzzle must contain 81 cells.
    if (cells.size() != SIZE * SIZE) {
        cerr << "Error: " << filename
             << " must contain exactly 81 Sudoku cells."
             << endl;
        return false;
    }

    // Convert the input characters into integers.
    // Zero represents an empty cell.
    for (int i = 0; i < SIZE * SIZE; i++) {
        int row = i / SIZE;
        int col = i % SIZE;

        if (cells[i] == '_') {
            original[row][col] = 0;
        } else {
            original[row][col] = cells[i] - '0';
        }

        grid[row][col] = original[row][col];
    }

    return true;
}

// Prints a Sudoku grid in a readable format.
void printGrid() const {
    cout << "+-------+-------+-------+" << endl;

    for (int row = 0; row < SIZE; row++) {
        cout << "| ";

        for (int col = 0; col < SIZE; col++) {
            if (grid[row][col] == 0) {
```

```
and newlines
        while (file >> ch) {
            if (ch == '_' || (ch >= '1' && ch <= '9')) {
                if (count < 81) {
                    int r = count / 9;
                    int c = count % 9;
                    grid[r][c] = ch;
                    count++;
                }
            }
        }
        // A valid 9x9 puzzle must contain exactly 81 elements
        return count == 81;
    }

    /**
     * @brief Prints the Sudoku board formatted with grid
lines for readability.
     */
    void print() const {
        for (int r = 0; r < 9; ++r) {
            if (r > 0 && r % 3 == 0) {
                std::cout << "------+-------+------\n";
            }
            for (int c = 0; c < 9; ++c) {
                if (c > 0 && c % 3 == 0) {
                    std::cout << "| ";
                }
                std::cout << grid[r][c] << " ";
            }
            std::cout << "\n";
        }
    }

    /**
     * @brief Checks if placing a number at a specific row
and column follows Sudoku rules.
     * @param row Row index (0-8)
     * @param col Column index (0-8)
     * @param num Character digit ('1'-'9') to check
     * @return true if the placement is valid, false otherwise.
     */
    bool isValid(int row, int col, char num) const {
        // Check the row
        for (int c = 0; c < 9; ++c) {
            if (grid[row][c] == num) return false;
        }

        // Check the column
        for (int r = 0; r < 9; ++r) {
```

```
                cout << "_ ";
            } else {
                cout << grid[row][col] << " ";
            }

            // Separate the 3x3 subgrids.
            if ((col + 1) % 3 == 0) {
                cout << "| ";
            }
        }

        cout << endl;

        // Draw horizontal boundaries between subgrids.
        if ((row + 1) % 3 == 0) {
            cout << "+-------+-------+-------+"
                 << endl;
        }
    }
}

// Checks whether a number already exists in a row.
bool isInRow(int row, int number) const {
    for (int col = 0; col < SIZE; col++) {
        if (grid[row][col] == number) {
            return true;
        }
    }

    return false;
}

// Checks whether a number already exists in a column.
bool isInColumn(int col, int number) const {
    for (int row = 0; row < SIZE; row++) {
        if (grid[row][col] == number) {
            return true;
        }
    }

    return false;
}

// Checks whether a number already exists in a 3x3 box.
bool isInBox(int startRow, int startCol,
             int number) const {
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (grid[startRow + row][startCol + col]
                == number) {
                if (grid[r][col] == num) return false;
            }

            // Check the 3x3 subgrid
            int startRow = (row / 3) * 3;
            int startCol = (col / 3) * 3;
            for (int r = 0; r < 3; ++r) {
                for (int c = 0; c < 3; ++c) {
                    if (grid[startRow + r][startCol + c] == num) return
false;
                }
            }

            return true;
        }

        /**
         * @brief Gets the character at a specific cell.
         */
        char getCell(int row, int col) const {
            return grid[row][col];
        }

        /**
         * @brief Sets the character at a specific cell.
         */
        void setCell(int row, int col, char val) {
            grid[row][col] = val;
        }
};

/**
 * @class SudokuSolver
 * @brief Manages the DFS backtracking algorithm to solve
Sudoku puzzles.
 */
class SudokuSolver {
private:
    SudokuBoard initialBoard;
    std::vector<SudokuBoard> solutions;
    std::string filename;

    /**
     * @brief Recursive helper function performing
Depth-First Search with Backtracking.
     * @param currentBoard Reference to the current state
of the board being explored.
     * @param row Current row index.
     * @param col Current column index.
     */
```

```
                return true;
            }
        }
    }

    return false;
}

// Returns true if placing a number in the specified
// empty cell does not violate Sudoku rules.
bool isSafe(int row, int col, int number) const {
    int boxRow = (row / 3) * 3;
    int boxCol = (col / 3) * 3;

    return !isInRow(row, number)
        && !isInColumn(col, number)
        && !isInBox(boxRow, boxCol, number);
}

// Checks whether the original puzzle contains
// conflicting given numbers.
bool hasValidInitialClues() {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            int number = grid[row][col];

            if (number == 0) {
                continue;
            }

            // Temporarily clear this cell so that its
            // value does not conflict with itself.
            grid[row][col] = 0;

            bool valid = isSafe(row, col, number);

            // Restore the original value.
            grid[row][col] = number;

            if (!valid) {
                return false;
            }
        }
    }

    return true;
}

/*
 * DFS with backtracking:
```

```
    void solveRecursive(SudokuBoard& currentBoard, int
row, int col) {
        // Base case: If we have successfully navigated past
the last row, a solution is found
        if (row == 9) {
            solutions.push_back(currentBoard);
            return; // Return to backtrack and search for
alternative solutions
        }

        // Determine coordinates for the next cell
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled, skip to the next
cell
        if (currentBoard.getCell(row, col) != '_') {
            solveRecursive(currentBoard, nextRow, nextCol);
        } else {
            // Try digits '1' through '9'
            for (char num = '1'; num <= '9'; ++num) {
                if (currentBoard.isValid(row, col, num)) {
                    currentBoard.setCell(row, col, num);          //
Choose
                    solveRecursive(currentBoard, nextRow,
nextCol); // Explore
                    currentBoard.setCell(row, col, '_');          //
Unchoose (Backtrack)
                }
            }
        }
    }

public:
    /**
     * @brief Construct a new Sudoku Solver object.
     * @param fname The puzzle input file name.
     */
    SudokuSolver(const std::string& fname) :
filename(fname) {}

    /**
     * @brief Initializes the solver by loading the puzzle from
file.
     * @return true if successful, false otherwise.
     */
    bool initialize() {
        return initialBoard.loadFromFile(filename);
    }
```

```
 *
 * 1. Find the next empty cell.
 * 2. Try numbers from 1 through 9.
 * 3. If a number is valid, place it in the cell.
 * 4. Recursively solve the remaining cells.
 * 5. Undo the placement to explore other possibilities.
 *
 * Unlike a solver that stops at the first solution,
 * this method continues searching to find every solution.
 */
void solveDFS() {
    int emptyRow = -1;
    int emptyCol = -1;

    // Find the first empty cell, scanning row by row.
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (grid[row][col] == 0) {
                emptyRow = row;
                emptyCol = col;
                break;
            }
        }

        if (emptyRow != -1) {
            break;
        }
    }

    // Base case: no empty cells remain, so a complete
    // valid solution has been found.
    if (emptyRow == -1) {
        solutionCount++;

        cout << "\nSolution #" << solutionCount
             << ":" << endl;

        printGrid();
        return;
    }

    // Explore every possible candidate for this cell.
    for (int number = 1; number <= SIZE; number++) {
        if (isSafe(emptyRow, emptyCol, number)) {
            // Choose: place a valid candidate.
            grid[emptyRow][emptyCol] = number;

            // Explore: recursively solve the next cell.
            solveDFS();
```

```
    /**
     * @brief Initiates the solving process.
     */
    void solve() {
        solutions.clear();
        SudokuBoard workingBoard = initialBoard;
        solveRecursive(workingBoard, 0, 0);
    }

    /**
     * @brief Displays the file name, initial puzzle, and all
found solutions or error messages.
     */
    void displayResults() const {
        std::cout <<
"========================================\n";
        std::cout << "Puzzle File Name: " << filename << "\n";
        std::cout <<
"========================================\n";
        std::cout << "Initial Puzzle:\n";
        initialBoard.print();
        std::cout << "----------------------------------------\n";

        if (solutions.empty()) {
            std::cout << "No solution found.\n";
        } else {
            std::cout << "Total Solutions Found: " <<
solutions.size() << "\n\n";
            for (size_t i = 0; i < solutions.size(); ++i) {
                std::cout << "Solution " << (i + 1) << ":\n";
                solutions[i].print();
                std::cout << "----------------------------------------\n";
            }
        }
    }
};

/**
 * @brief Main execution function.
 */
int main(int argc, char* argv[]) {
    std::string filename;

    // Accept filename via command-line argument or prompt
user interactively
    if (argc > 1) {
        filename = argv[1];
    } else {
        std::cout << "Enter the Sudoku puzzle file name: ";
        std::cin >> filename;
```

```
            // Undo the choice (backtrack) so that
            // other candidate numbers can be tested.
            grid[emptyRow][emptyCol] = 0;
        }
    }
}

public:
    // Constructor initializes an empty puzzle.
    Sudoku() : solutionCount(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                original[row][col] = 0;
                grid[row][col] = 0;
            }
        }
    }

    // Loads, displays, and solves a puzzle file.
    void processFile(const string& filename) {
        cout <<
"\n========================================"
             << endl;
        cout << "Puzzle file: " << filename << endl;
        cout <<
"========================================"
             << endl;

        if (!loadFromFile(filename)) {
            return;
        }

        cout << "\nInitial puzzle:" << endl;
        printGrid();

        // Reject contradictory starting clues before DFS.
        if (!hasValidInitialClues()) {
            cout << "\nNo solution found" << endl;
            return;
        }

        // Search for all possible solutions.
        solutionCount = 0;
        solveDFS();

        // Report the outcome if no valid grid was found.
        if (solutionCount == 0) {
            cout << "\nNo solution found" << endl;
        } else {
            cout << "\nTotal solutions found: "
    }

    // Instantiate the solver object
    SudokuSolver solver(filename);

    if (!solver.initialize()) {
        std::cerr << "Error: Could not open or read puzzle file
'" << filename << "' properly.\n";
        return 1;
    }

    // Solve and output results
    solver.solve();
    solver.displayResults();

    return 0;
}
```

```
            << solutionCount << endl;
        }
    }
};

// Program entry point.
int main(int argc, char* argv[]) {
    vector<string> filenames;

    // If filenames were provided as command-line
arguments,
    // collect them for processing.
    if (argc > 1) {
        for (int i = 1; i < argc; i++) {
            filenames.push_back(argv[i]);
        }
    } else {
        // Otherwise, let the user enter filenames.
        cout << "Enter Sudoku puzzle filenames "
             << "(one per line)." << endl;
        cout << "Press Enter on an empty line to finish."
             << endl;

        string filename;

        while (true) {
            cout << "Filename: ";

            if (!getline(cin, filename) || filename.empty()) {
                break;
            }

            filenames.push_back(filename);
        }
    }

    if (filenames.empty()) {
        cout << "No puzzle files were provided." << endl;
        return 0;
    }

    // Create a separate Sudoku object for each puzzle.
    for (size_t i = 0; i < filenames.size(); i++) {
        Sudoku puzzle;
        puzzle.processFile(filenames[i]);
    }

    return 0;
}
```